Agile Project Management

Chapter 2: Product Definition

Chapter Objectives

In this chapter, we will:

- Review the concepts of defining the product
- Introduce the concept of user stories, epics, themes, and product backlogs
- Construct a product backlog

Chapter Concepts

User Stories, Epics, and Themes

The Product Backlog

Chapter Summary

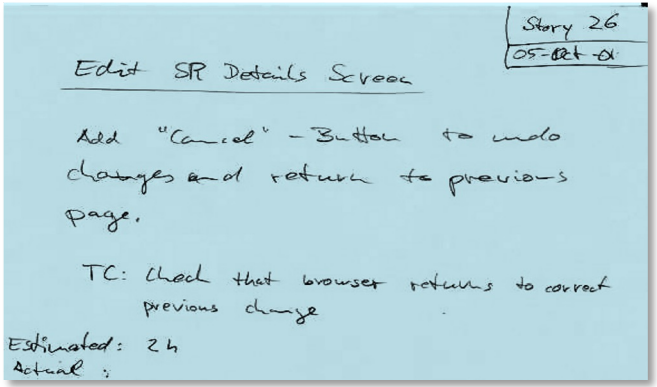
User Stories

- A user story is a very high-level definition of a requirement
  - Contains enough information so that the developers can produce a reasonable estimate of the implementation effort
- Each user story is a short description of the behavior of the system from the point of view of the user of the system*
  - Each story describes one thing that the system needs to do
  - In XP, the system is specified entirely through stories*
- A reminder to have a conversation with your customer [Scott Ambler]
- User stories belong to the customer (the one with the requirements)

*Ron Jeffries, *Extreme Programming Installed*, Addison-Wesley, 2000, p. 37

User Stories on Cards

- Tangible unit of:
  - Discussion
  - Scheduling
  - Estimation
  - Functional testing
  - Completion
- The cards are tokens for conversations
- The cards represent stories as scheduled and completed
- The cards aren't official documents



Source: Ron Jeffries, personal communications

Concise Story Statement

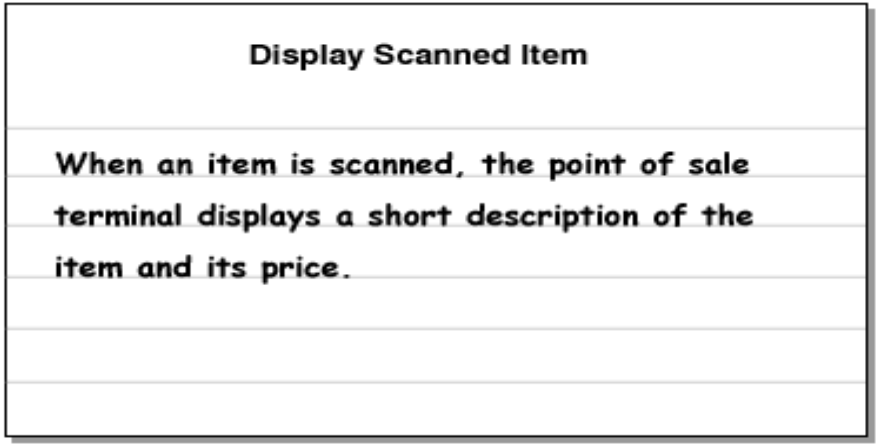
- User stories must be:
  - Testable
  - Estimable
  - Therefore: clear, precise, concise
- As a [type of user] I want [some particular feature] so that [some benefit is received]*
  - Example:

    As a bank customer I want to view my current account balance so that I know my recent deposit went through
- Or, "I would like to <do this with the system> so that I can <accomplish this business objective>"

*Source: Rachel Davies

Discussion: User Story Card

- If you were a developer and received this card with a stakeholder's user story on it, could you sit down and start producing code to satisfy the request?
- If not, what do you need to know in order to complete the task successfully?



INVEST Guideline for User Stories

- **Independent** from each other so that they can be prioritized easily
- **Negotiable** so that scope can be defined together with the customer
- **Valuable** so that each one is considered useful by the customer
- **Estimatable** so that prioritization can take into account costs and cycle times
- **Small** so that they don't stay in development forever, impacting on iterativity
- **Testable** so that we know when we're finished

Exercise 2.1: User Stories

- Read the case study information
- Individually, write as many user stories as you can to represent what the customers and business stakeholders want out of the Buy Local Bonds application
  - Write the stories on index cards
- In your workgroups, collect and compare all the user stories to create a single deck of stories

Epics

- Stories may start out as high-level requests which are broken down
- Stories may be aggregated into groups for ease of understanding and management, which are generally called “epics”

Example of Epics

- Epic from case study:
  - "Find bonds that fund projects in my area"
  - The epic may be written "As a Meridian Financial customer I want to see which local projects a bond funds so I can decide whether to invest in my own community"
  - User Story for this epic might be "As a Meridian Financial customer I want a list of bonds issued by my school district in the past twelve months"

Exercise 2.2: Write Epics

- In your workgroups, write a set of epics for the case study
  - You may already have some epics defined from the first user story exercise
- Make an attempt to cover all the needs that the business people expressed with some epic

Exercise 2.3: Epics to Stories (optional)

- Look at the set of epics you created for your case study in your workgroup
- Pick three epics and break them down into user stories
- Make the user stories as small as you can
- Make sure that everyone on the team agrees to the stories
- At this point, you should have a set of epics and a set of user stories, some of which are associated with an epic

Themes

- Themes are related to release and identify the overall set of stories scheduled for the next release
- Themes may be functional or non-functional in nature

Example Themes

- User stories:
  - As a Buy Local Bonds user I want pull down menus to choose actions so that I don't have to keep going back and forth between screens"
  - As a Buy Local Bonds user I want to filter bonds by maturity without re-running my search so that I do not lose my place
  - As a Buy Local Bonds user I want to enter a zip code and have the system find my local issuers so I don't have to know their names"
- Theme: Improve User Experience or Improve User Interface

Exercise 2.4: Define Themes for the Case Study

- Revisit the initial set of user stories from the earlier exercise
- In your workgroups, assemble the stories into themes
- Make a list of themes on the flip chart

Chapter Concepts

User Stories, Epics, and Themes

The Product Backlog

Chapter Summary

Product Backlog

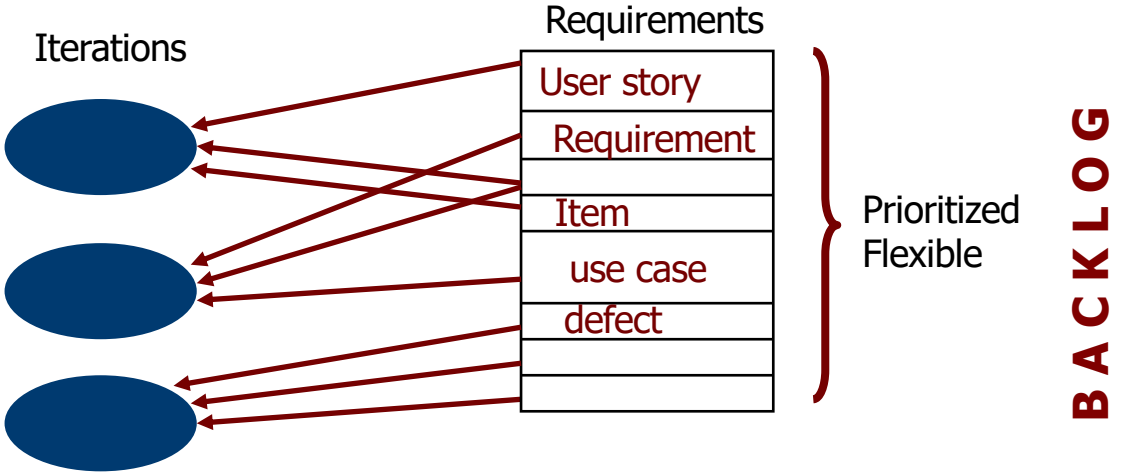
- The product backlog is an ordered list of everything that might be needed in the product and is the single source of requirements for any changes to be made to the product
- The product owner is responsible for the product backlog, including its content, availability, and ordering
- Product backlog is never complete
  - Exists as long as the product exists
- Starts with initially known and best understood requirements
- The product backlog lists all features, functions, requirements, enhancements, and fixes that constitute the changes to be made to the product in future releases

Source: Scrum Guide

Non-Functional Requirements

- User stories are functional
- Writing non-functional requirements as user stories is awkward and strained
- Non-functional requirements, including constraints, can be written as items on the project backlog

Product Backlog (continued)



- Requirements on the stack may be selected from the top
  - Combined into functional groups of requirements
- Requirements delivered as separate
  - Releases
  - Phases
  - Iterations

Exercise 2.5: Backlogs

- In your workgroups, create a product backlog for the case study
  - Include on the product backlog all functions and features that the product owner would request on behalf of the users
  - Include any dependent tasks that are necessary to complete the functions and features for the product owner
    - E.g., building a database
  - Include any test or QA activities associated with completing the job
- Assign a business value to each item on the product backlog
  - Use a ranking of H. M. L.
- Prioritize the product backlog in business value order with the highest value item at the top
  - Prioritize all items on the backlog

Chapter Concepts

User Stories, Epics, and Themes

The Product Backlog

Chapter Summary

Chapter Summary

In this chapter, we have:

- Reviewed the concepts of defining the product
- Introduced the concept of user stories, epics, themes, and product backlogs
- Constructed a product backlog